IDEA

Integrated Development Environment for COSMIC Software

C Compiler and Zap Debugger

Quick Start Guide

PC/Windows XP/7/8/10

Document Version V2.1 Jan. 2016

Copyright © COSMIC Software Inc. 1999, 2016

All Trademarks are the property of their respective owners

IDEA Quick Start Guide

- Overview
- Running IDEA
- Managing a project
- Building a project
- Debugging a project

© 2016 COSMIC Software

Overview

What is IDEA?

IDEA is an integrated development environment and editor for managing cross development projects using Cosmic tools. IDEA is supplied in a target specific version customized to the Cosmic tools you are using. For example, IDEACXM is designed for use with the Cosmic cross compiler and ZAP debugger for the Cortex-M processor. In order to run IDEA you must have the matching cross compiler installed on your system; the ZAP debugger is optional.

With IDEA, you can create and edit projects; compile, assemble and link C or assembler code; run an application Make or Build; and run a ZAP debugger session; all with a few simple mouse clicks in a user-friendly, graphical Windows interface.

Using IDEA

The IDEA GUI (Graphical User Interface) provides immediate access to all the tools you need to manage full projects.

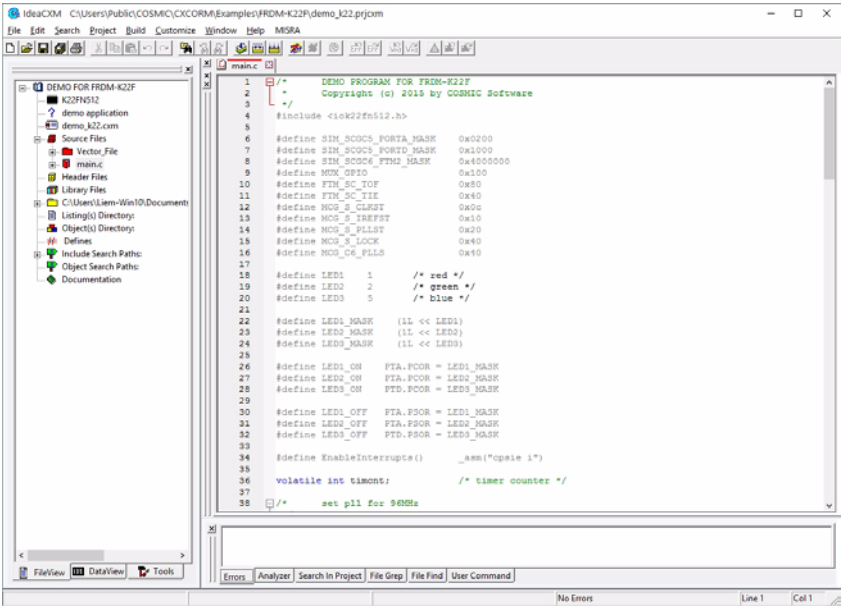


Figure 1: IDEA GUI

The **Project window** at the left provides a graphical, tree-structured view of your project. Using just the Project window, you can add or remove files from the project, set compiler options, configure build utilities, and much more.

The **File windows** at the right allow you to open project files for editing and compiling. IDEA provides color-coding of Comments, Preprocessor Keywords, C Keywords, and several other coding items so that you can easily edit source code files.

The **Output Pane windows** at the bottom provides different outputs: compiler and linker errors or warnings, results of the analyzer, results of the different searchs.

© 2016 COSMIC Software

Running IDEA

Starting IDEA

From the Windows Start menu, select **Programs>Cosmic Tools>CXCORM** or double click on the on the desktop.

The IDEA main window appears:

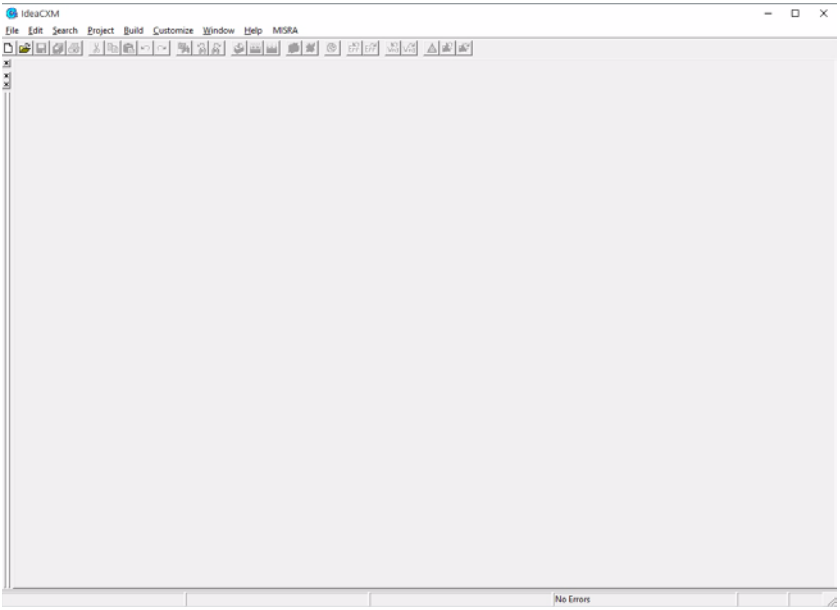


Figure 2: IDEA main window

After you open a project (**Project>Load**) and some files within the project, the IDEA main window appears as in the following figure:

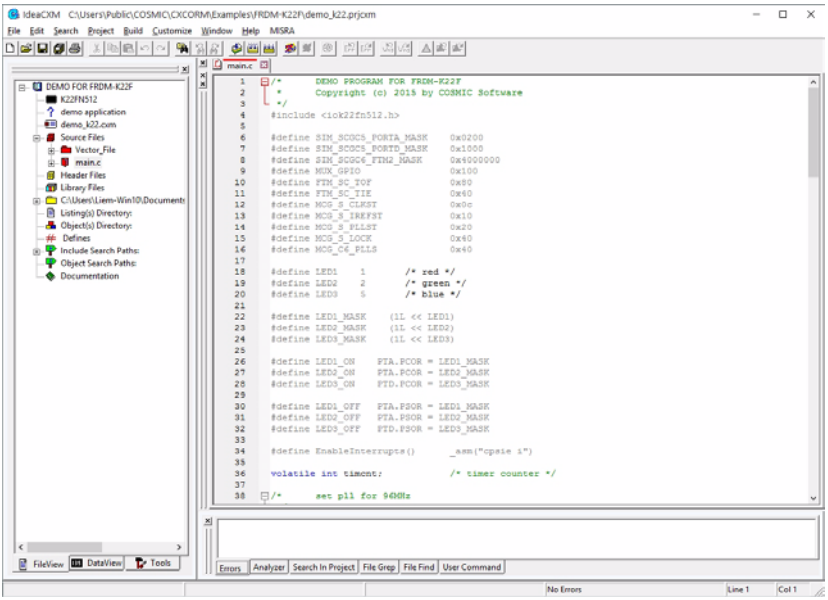


Figure 3: IDEA main window with Project and File windows open

The IDEA main window is the principal graphical user interface (GUI) for the program.

For complete details on the components of the IDEA main window, refer to Chapter 4, "*IDEA User Interface*" in the IDEA User's Guide.

Getting Help

The **Help** drop-down menu provides help on the C language and on the C Library. You can also view IDEA version information.

NOTE

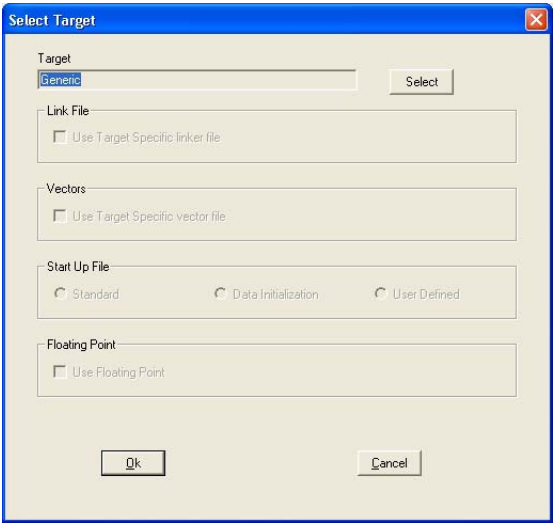
On some versions of Windows 7/8 and 10, you may get the display of the Windows Help instead of the Idea Help. In this case, you may need to update Winhlp32.exe. For that, go and visit the Microsoft web site.

Open the **Help** drop-down menu by clicking on **Help** in the Main menu. Alternatively, type **Alt+H**.

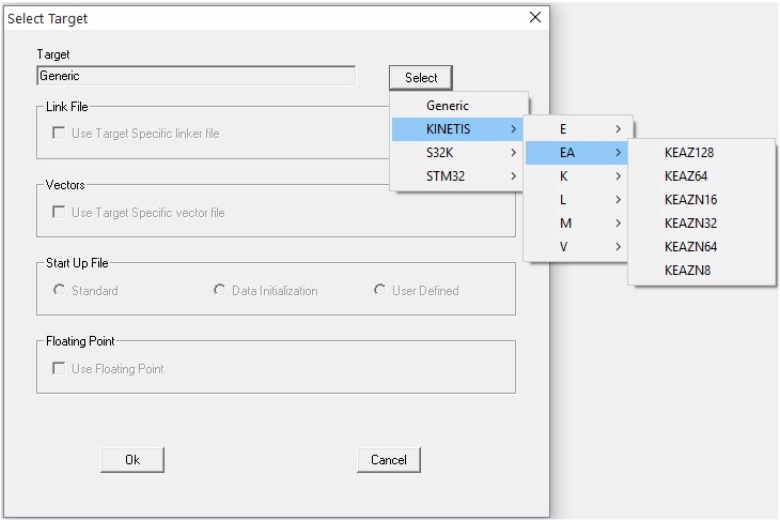
© 2016 COSMIC Software

Starting a new project

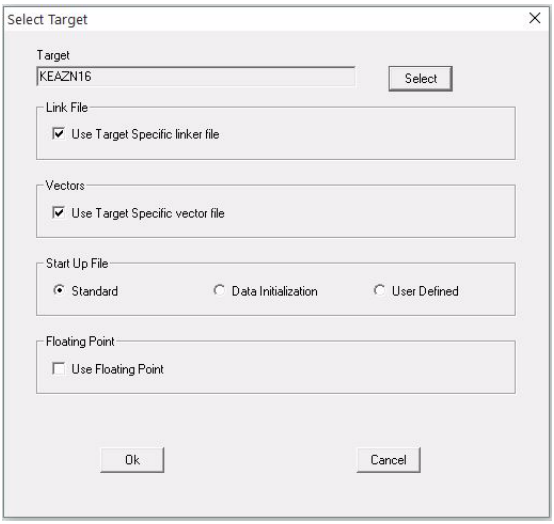
To start a new project, select **Project>New Application** from the Main menu. The **Select Target** window is displayed:



By default, a “generic” target is selected. Click on the **Select** button to select your target allowing thus automatic file selections: link file, vector file.



For example, selecting a specific target, you will get:



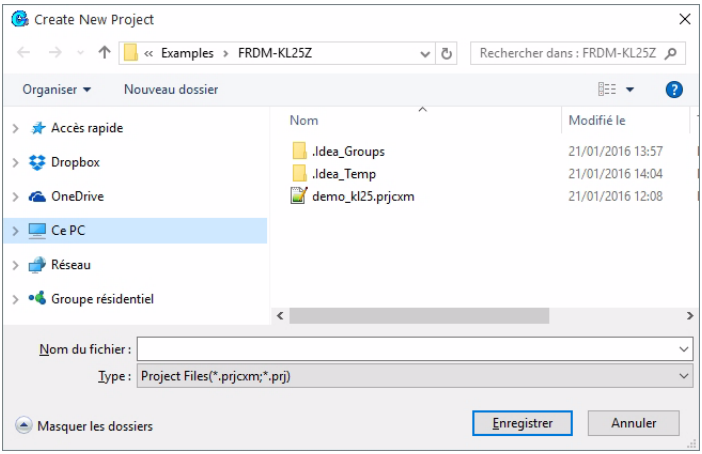
In this configuration, when the **Link File** checkbox is selected, the link file, the vector file and the startup file are automatically selected and

© 2016 COSMIC Software

configured for the specified target. Note that you cannot modify the link file. You can choose to use the standard startup file, with or without the automatic data initialization, or use your own startup file. In **automatic mode**, the linker command file is automatically recreated each time the project is modified by either adding a new source file or changing the compiler options.

You prefer to use and modify your own link file: uncheck the "*Use target Specific Linker file*".

Finally, click on the OK button and save your project:



The Project window appears with a new project opened.

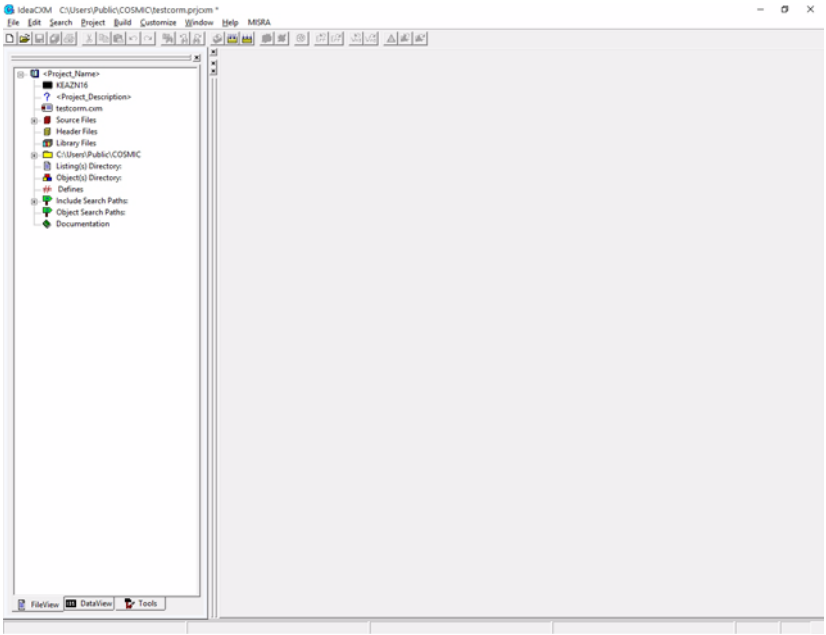


Figure 4: IDEA Project window with new project open

The Project window displays the various project components as icons in a tree-structured format, similar to Windows Explorer. Each icon in the project tree represents a project component.

Project Components

	Project Name
	Project Target
	Project Description
	Project Target File Name
	Project Source Files
	Project Header Source Files
	Project Library Files

© 2016 COSMIC Software

	Project Directory
	Project Listing(s) Directory
	Project Object(s) Directory
	Project Defines
	Project Include Search Paths
	Project Object Search Paths
	Project Documentation

A ⊞ sign next to a component icon means that sub-components are hidden below the icon. Click on the ⊞ sign to display the sub-components, or if it is a file, double click on the icon to open it.

A ⊟ sign next to a component icon means that the first level of sub-components below the icon is displayed. Click on the ⊟ sign to hide the sub-components.

For additional details on the Project window, refer to Chapter 4, "*IDEA User Interface*" in the IDEA User's Guide.

For details on project management, refer to Chapter 7, "*Managing an IDEA Project*" in the IDEA User's Guide.

Exiting IDEA

To exit IDEA, click on **Exit** in the **File** menu. Alternatively, type **Alt+F+X**.

If you have selected **Auto Save before C/asm** in the **Customize** drop-down menu (**Alt+C+A**), all changed files are saved prior to exiting. If you have not selected **Auto Save before C/asm**, a dialog box appears in turn for each changed file and lets you select whether to save the file or not.

Managing a Project

Opening the project

IDEA is supplied with an example project, **acia.prjcxm** for the Cosmic Cortex-M compiler. You can use this example project to become familiar with the principles of managing an IDEA project.

If you opened a new project earlier, save it by clicking on **Project> Save** from the main menu. Close the project by selecting **Project> Close** from the Main menu.

To open the example project, select **Project->Load** in the Tool bar. In the dialog box that appears, select **acia.prjcxm** from the **Examples** folder. The Project window appears with the **acia.prjcxm** project opened.

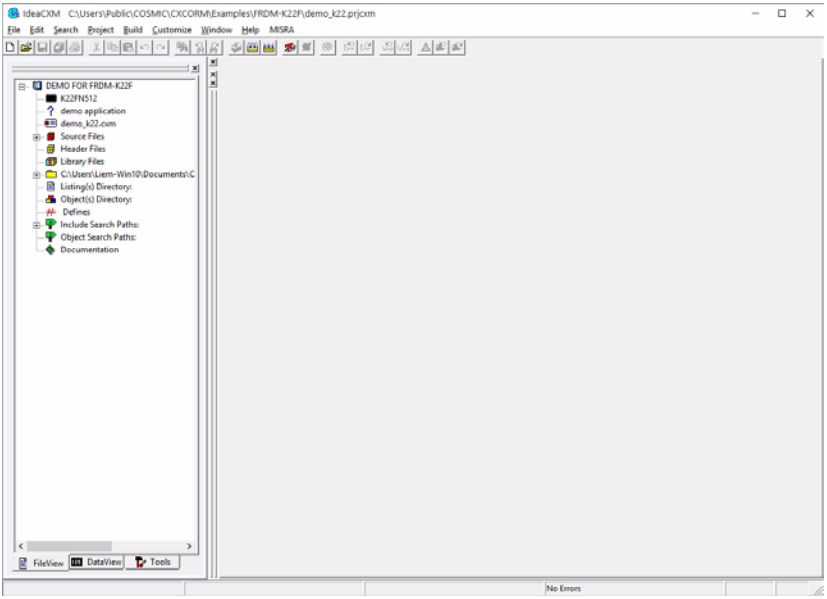


Figure 5: Project pane with demok22.prjcxm project opened

© 2016 COSMIC Software

Naming the project

The **Project Name** component lets you specify a name for the project. It also represents the parent component for all project sub-components.

To specify a name for the project, click on the **Project Name** icon . A text cursor appears to the right of the icon. Click on the text cursor to open a text box and enter a project name.

Right click on the **Project Name** icon to view a menu of project commands. These commands are shown in the following table.

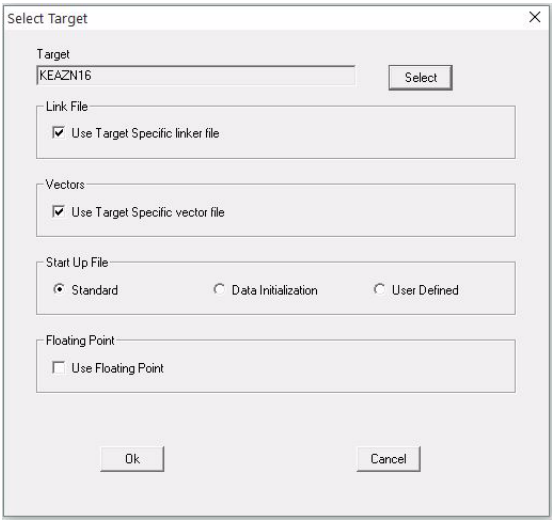
Add File	Adds a source file to the project.
Add Group	Adds a group of file to the project.
Save	Saves the project.
Save As	Saves the project with a new name.
Make	Checks source file up-to-date status and dependencies. Then selectively compiles or assembles any out-of-date files and runs the Linker. The icons in the **Project Source Files** folder are colored yellow.
Build	Performs a **Make** as described above and then runs any utilities selected in the **Builder Configuration** dialog box. To have the **Build** rebuild all files regardless of their up-to-date status, right click on the project name, select **Mark All**, and then run the Builder.
Build All	Performs a **Make** regardless of their up-to-date status and then runs any utilities selected in the **Builder Configuration** dialog box. To have the **Build** rebuild all files, right click on the project name, select **Build All**, and then run the Builder.
Mark All	Marks all project source files, including groups and sub groups, for recompile/assemble without changing the file time/date stamp. The icons in the **Project Source Files** folder are colored orange.

Touch All	Marks all project source files, including groups and sub groups, for recompile/assemble and updates all project source files, including groups and sub groups, with the current system date and time stamp. The icons in the **Project Source Files** folder are colored red.
Documentation	Adds a document file to the project.

Managing the Project Target Processor

The **Project Target** component lets you specify the specific target for your project, allowing thus the linker, vector and startup files to be set up accordingly.

To specify the target for the project, right click on the **Project Target** icon to display the project target selection:



By default, an automatic link file, a specific and matching target vector file are created. You can also decide which startup file you want to use (with or without automatic data initilization or your own), using or not floating point libraries (double precision or single precision library will be then set according to the **+sprec** compiler option setting). Note that

© 2016 COSMIC Software

the linker command file is then only accessed in read-only mode. It is automatically recreated each time the project is modified by either adding a new source file or changing the compiler options.

You prefer to create your own link file, vector file and startup file: uncheck the "**Link File**" checkbox.

Describing the project

The **Project Description** component lets you specify a short description for the project.

To specify a description for the project, click on the **Project Description** icon ?. A text cursor appears to the right of the icon. Click on the text cursor to open a text box and enter a short project description.

Naming the project target file

The **Project Target File Name** component lets you specify a target file name for the project (for example, **acia.cxm**. This name is used as the root name for the linked executable.

To specify a project target file name, click on the **Project Target File Name** icon .

A text cursor appears to the right of the icon. Click on the text cursor to open a text box and enter a target file name. Be sure to include the target file name extension; for example, "**.cxm**" for the Cosmic Cortex-M compiler.

Right click on the **Project Target File Name** icon to view a menu containing target file commands. These commands are shown in the following table.

Inspect Object	Runs the **Object Inspector** utility (***cobj***) on the target file.
Show Debug	Runs the **Debug Info Examiner** utility (*cprd*) and opens the project debug file in read-only mode.

Produce Hex Records	Runs the **Hex Converter** utility (*chex*), which translates executable images produced by the *clnk* linker to one of several hexadecimal interchange formats.
Produce Absolute Listings	Runs the **Absolute Lister** utility (*clabs*) to generate absolute listings.
Produce IEEE Output	Runs the **IEEE695 Converter** utility (*cv695*) to generate IEEE695 debug format.
Produce ELF Output	Runs the **ELF/DWARF Converter** utility (*cvdwarf*) to generate ELF/Dwarf debug format.
Debug File	Runs the selected ZAP debugger and loads the linked executable.
Delete	Deletes the project target file. A pop-up dialog box asks you to confirm the deletion

Managing project source files

The **Project Source Files** component lets you specify the C and Assembly language source files to be included in the project.

Right click on the **Project Source Files** icon (or) to view a menu containing source file management commands. These commands are shown in the following table.

Add File	Adds a source file to the project.
Add Group	Adds a group of files to the project.
Touch All	Updates all project source files, including groups and sub groups, with the current system date and time stamp and marks them for recompile/assemble when a **Make** or **Build** is executed. The icons in the **Project Source Files** folder are colored red.
Touch Files Only	Updates all project source files present at the root level, not including groups and sub groups, with the current system date and time stamp and marks them for recompile/assemble when a **Make** or **Build** is executed. The icons in the **Project Source Files** folder are colored red.

© 2016 COSMIC Software

Mark All	Marks all project source files for recompile/assemble, including groups and sub groups, when **Make** or **Build** is executed. This option does not change the time-date stamp of the files. The icons in the **Project Source Files** folder are colored orange.
Mark Files Only	Marks all project source files present at the root level, not including groups and sub groups, for recompile/assemble when **Make** or **Build** is executed. This option does not change the time-date stamp of the files. The icons in the **Project Source Files** folder are colored orange.

Adding source files to the project

You can add source files to the project using the **Add File** command:

- To add a source file using the **Add File** command, right click on the **Project Source Files** icon, select **Add File**, and select the file to add from the **Add File** dialog box.

You can select more than one source file at a time using standard Windows conventions for selecting and grouping files.

Working with individual source files

Each source file included in the project is listed next to a **Source File** icon . The **Source File** icon lets you view the source file and its attributes.

Right click on the **Source File** icon to view a menu containing source file commands. These commands are shown in the following table.

Edit	Open the source file in edit mode.
Load (read only)	Open the source file in read-only mode.
Compile	Compile or assembles the source file. The source file icon is colored yellow if the **Compile** is successful.
Mark	Mark the source file for rebuilding. The **Source File** icon is colored orange.

Touch	Update the source file with the current system date and time stamp and marks it for rebuilding. The **Source File** icon is colored red and the **Source File Time Stamp** icon is updated with the new date and time.
Ignore	The file is in the project, but it is ignored during a Build/make process.
Replace with backup	When a file is edited, a backup is made, allowing thus the file to be restored with the previous version.
Remove	Remove the source file from the project.
Properties	Open the **File properties** dialog box, where you can see the file system properties, its dependencies, and specify compiler define options for the source file.
Options	Open the **Compiler** (or Assembler) **Options for Source File** dialog box, where you can specify options for the source file.
Define	Open the **#defines** dialog box, where you can specify compiler define options for the source file.
Documentation	Add a document file for the source file.

Each icon in the source file tree represents a source file component. The source file components are described in the following table.

	Source File Time Stamp
	Source File Documentation
	Source File Options
	Source File Defines
	Source File Dependencies
	Source File Functions. Appears only if **Project Analysis** option is selected in **Options** sub-menu.
	Source File Variables. Appears only if **Project Analysis** option is selected in **Options** sub-menu.

© 2016 COSMIC Software

Source File Time Stamp

The **Source File Time Stamp** component and icon shows the day, date, and time that the file was last saved or “touched”.

Source File Documentation

The **Source File Documentation** component shows all documents that are associated with the source file.

Right click on the **Documentation** icon and select **Add Doc** to associate a documentation file with the source file.

The **Document** component lets you view, edit, or remove a document associated with a source file. The appearance of the **Document** icon varies, depending on the type of document.

Right-click on the **Document** icon to view a menu containing documentation file commands. The documentation file commands are described in the following table.

Load (read only)	Open the document in read-only mode.
Open	Open the document for editing using the appropriate Windows-registered application.
Remove	Remove the document from the project.

Source File Options

The **Source File Options** component lets you specify compiler or assembler options for the source file. These options override the default project compiler or assembler options.

Right-click on the **Source File Options** icon to open the **Compiler** (or **Assembler**) **Options for Source File** dialog box.

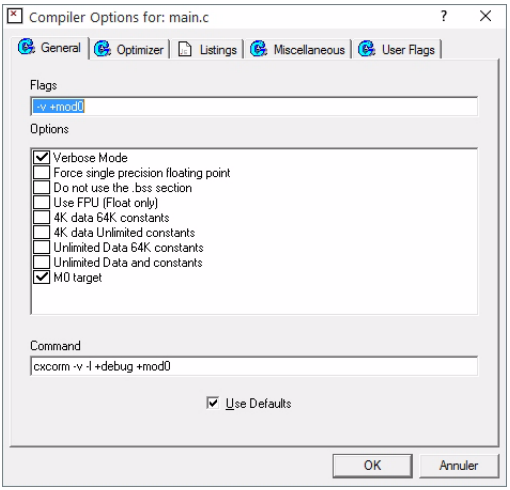


Figure 6: Compiler Options for Source File dialog box

The **Compiler** (**Assembler**) **Options** dialog box has five tabs:

- **General options**
- **Optimizer options**
- **Listings options**
- **Miscellaneous options**
- **User Flags**

Choose a tab and select the desired options. To select an option simply click on the option description. To deselect an option click on the option again.

The source file compiler or assembler options will override the default project compiler or assembler options. Selecting the bottom checkbox restores the options to the default setting selected at the **Tool** level.

Refer to “Setting default compiler options” on page 32 or “Setting default assembler options” on page 34 for details.

© 2016 COSMIC Software

Source File Defines

The **Source File Defines** component lets you specify compiler #define options for a source file.

Right-click on the **Source File Defines** icon # to open the **#defines** dialog box and specify up to twenty user-defined preprocessor symbols.

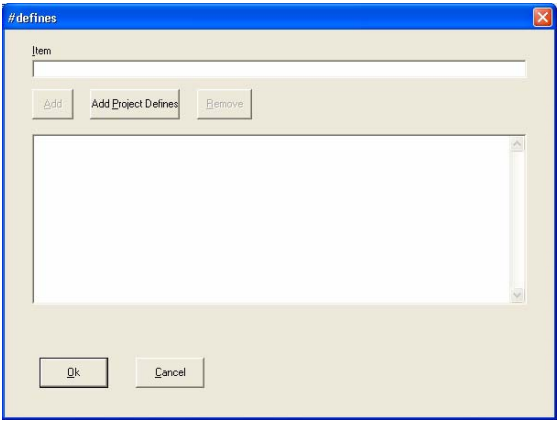


Figure 7: Source File #defines dialog box

To add a symbol to the list, enter the symbol in the **Item** field and click on **Add**. To remove a symbol from the list, select the symbol and click on **Remove**.

You can also add project #defines to the source file #define list by clicking on **Add Project Defines**.

Refer to “The Folder icon shows folders in the project directory. You can then browse and set the new working directory.” on page 25 for details.

After you add #defines, they appear as individual sub-components in the **Defines** list, each one after a **Define** icon DEFINE. The define symbol is shown to the right of the icon. In addition, the day, date, and time that the #defines were last updated is shown next to the **Source File Defines** icon.

Source File Dependencies

The **Source File Dependencies** component and icon let you view the files that are named in the source file #includes.

The **File** icon shows a file that is named in the source file #includes. If the file name is enclosed in brackets, it is a system include file and is not typically modified in each project. If the file name is enclosed in quotes, it is a user include file and can be modified. Right-click on the **File** icon to display a menu include file commands. The include file commands are described in the following table.

Open	Execute any associated program
Edit	Open the file for editing.
Load (read only)	Open the file in read-only mode.
Touch	Update the file with the current system date and time and marks it for rebuilding.
Delete	Delete the file from the folder and put it in the Windows trash.
Properties	Output the file properties.

Source File Functions

The **Source File Functions** component and icon let you view the functions in the source file. The **Function** icon shows a function in the source file. Double click on the function name to open the source file at the function or right-click on the **Function** icon and select **Go to Definition** to display the function in the source file or see its **Properties**.

Source File Variables

The **Source File Variables** component and icon let you view the variables that are local to the source file.

The **Variable** icon shows a variable declared with the source file. Double click on a variable name to open the source file at its declaration or right-click on the **Variable** icon and select **Go to Definition** to display its definition in the source file or see its **Properties**.

© 2016 COSMIC Software

NOTE

Functions *and* ***Variables*** *appear in the Project window only if* ***Project Analysis*** *is selected from the* ***Options*** *menu.*

Specifying the project directory

The **Project Directory** component is used to set the working directory for the project. This is typically where the source code files for the project are located.

Right-click on the **Project Directory** icon and select **Update** to open the **Set Working Directory** dialog box and set the path to the source files.

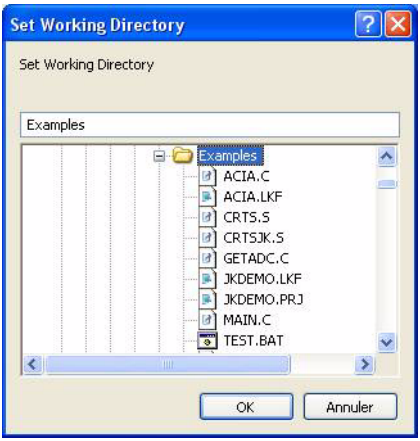


Figure 8: Path Editor dialog box

The **Folder** icon shows folders in the project directory. You can then browse and set the new working directory.

Specifying Listings directory

The **Listings Directory** component lets you specify listings path for the compiler (**-cl** option). Default is to set listings in the source project folder.

Specifying Objects directory

The **Objects Directory** component lets you specify objects path for the compiler (**-co** option). Default is to set objects in the source project folder.

Specifying Project Defines

The **Project Defines** component lets you specify #define options for the project.

Right-click on the **Project Defines** icon # to open the #defines dialog box and specify up to twenty preprocessor symbols.

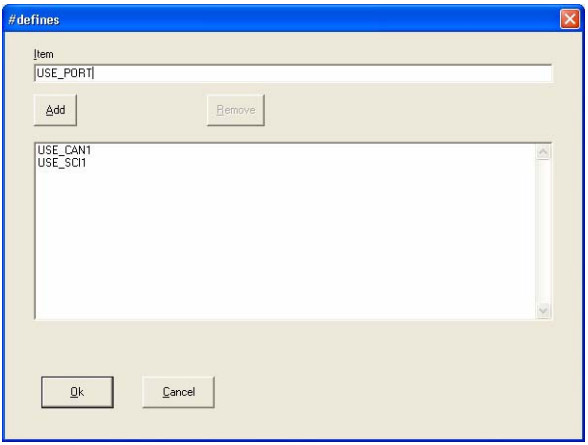
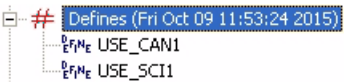


Figure 9: Project #defines dialog box

To add a symbol to the list, enter the symbol in the **Item** field and click on **Add**. To remove a symbol, select it and click on **Remove**.

Symbols defined as project #defines can be imported into source file #defines. Refer to “Source File Defines” on page 23 for details.

After you add project #defines, they appear as individual sub-components in the **Defines** list, each one after a **Define** icon DEFINE:



© 2016 COSMIC Software

The define symbol is shown to the right of the icon. In addition, the day, date, and time that the project #defines were last updated is shown next to the **Project Defines** icon.

Specifying project Include Paths

The **Project Include Search Paths** component lets you specify include paths for the compiler (**-i >** option).

Right-click on the **Project Include Search Paths** icon , select **Update** to open the **Include Path Editor** dialog box then click on the **Select** button and specify up to twenty include paths for the project.

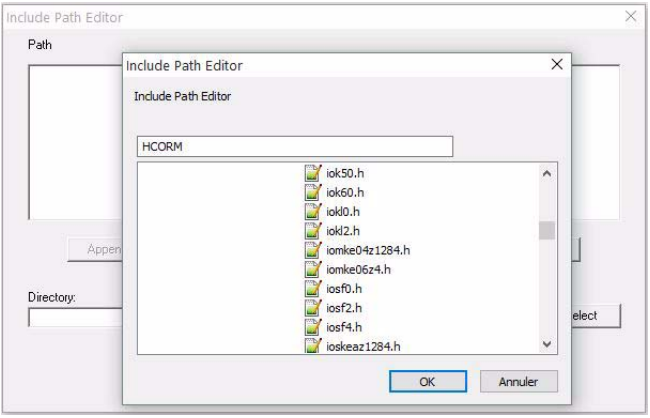


Figure 10: Include Search Path Editor

You can specify paths in any desired order. The paths are searched from top to bottom by the compiler.

You can use the **Drives** and **Directory** fields to specify the include path. Click on **Append** to add the path to the bottom of the list in the **Path** field.

To position the new path before or after the selected path, select an include path in the **Path** field and click on **Add Before** or **Add After**.

After you add include paths, they appear in order next to the **Project Include Paths** icon and as components in the **Project Include Paths** list, each one after a **Folder** icon.

Include path folders and files

The **Include Path Folder** icon shows folders for include file paths in the project directory. The **Folder** icon shows folders in an include file path.

The **File** icon shows files in an include file path. Right-click on the **File** icon to display a menu with the following commands:

Open	Execute any associated program.
Edit	Open the file for editing.
Load (read only)	Open the file in read-only mode.
Touch	Update the file with the current system date and time and marks it for rebuilding.
Delete	Delete the file from the folder and put it in the Windows trash.
Properties	Output the file properties.

Specifying project Object paths

The **Project Object Search Paths** component lets you specify object paths for locating object files in order to check if the objets are up to date. Default is to set objects in their source folders. The **object search paths** list is automatically updated when defining an explicit object directory.

Right-click on the **Project Object Search Paths** icon , select **Update** to open the **Object Path Editor** dialog box then click on the **Select** button and specify up to twenty include paths for the project.

© 2016 COSMIC Software

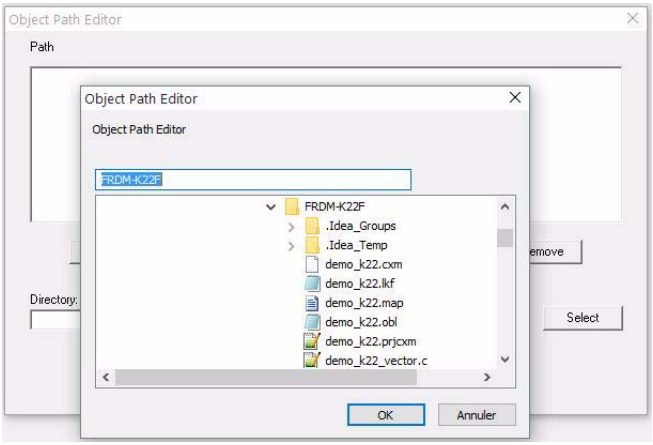


Figure 11: Object Search Path Editor

You can specify paths in any desired order. The paths are searched from top to bottom by the compiler.

You can use the **Drives** and **Directory** fields to specify the object path. Click on **Append** to add the path to the bottom of the list in the **Path** field.

To position the new path before or after the selected path, select an object path in the **Path** field and click on **Add Before** or **Add After**.

After you add object paths, they appear in order next to the **Project Object Paths** icon and as components in the **Project Object Paths** list, each one after a **Folder** icon.

Specifying project documentation

The **Project Documentation** component shows all documents that are associated with the project.

Right click on the **Documentation** icon and select **Add Doc** to associate a documentation file with the project.

The **Document** icon lets you view, edit, or remove a document associated with the project. The appearance of the icon varies, depending on the type of document.

Right-click on the **Document** icon to view a menu containing documentation file commands. These commands are described in the following table.

Load (read only)	Opens the document in read-only mode.
Open	Opens the document for editing.
Remove	Removes the document from the project.

© 2016 COSMIC Software

Configuring Project Tools

The **Project Tools** tab lets you set project default options for:

- **Compiler**
- **Assembler**
- **Linker**
- **Builder**
- **Debugger**

The **Builder** component lets you configure build utilities for the project, including:

- **Object Inspector (*cobj*)**
- **Hex Converter (*chex*)**
- **Debug Info Examiner (*cprd*)**
- **Absolute Lister (*clabs*)**
- **IEEE-695 Converter (*cv695*)**
- **ELF/Dwarf Converter (*cvdwarf*)**

Setting default compiler options

The **Compiler** component lets you set the default compiler options that are used to compile all C code **(.c)** files in a project.

Right click on the **Compiler** icon and select **Option** to open the **Compiler Options** dialog box. You can also double-click on the **Compiler** icon to display the **Compiler Options** icon and then right-click on the **Compiler Options** icon to open the **Compiler Options** dialog box.

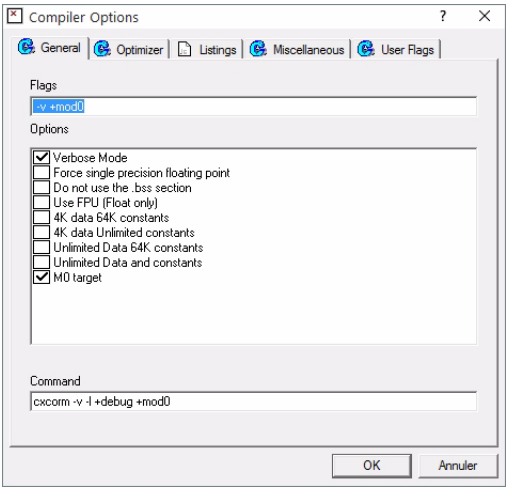


Figure 12: Compiler Options dialog box

The **Compiler Options** dialog box has five tabs:

- **General options**
- **Optimizer options**
- **Listings options**
- **Miscellaneous options**
- **User Flags**

© 2016 COSMIC Software

Choose a tab and select the desired options. Selected options are displayed in bold and unselected options are greyed out. To select an option, simply click on the option description. To deselect an option, click on the option again.

The default compiler options can be overridden by setting compiler options for the individual source files. Refer to “Source File Options” on page 21 for details.

Setting default assembler options

The **Assembler** component lets you set the default assembler options that are used to assemble all assembly language (**.s**) files in a project.

Right click on the **Compile** icon ASM and select **Option** to open the **Assembler Options** dialog box. You can also double-click on the **Assembler** icon to display the **Assembler Options** icon and then right-click on the **Assembler Options** icon to open the **Assembler Options** dialog box.

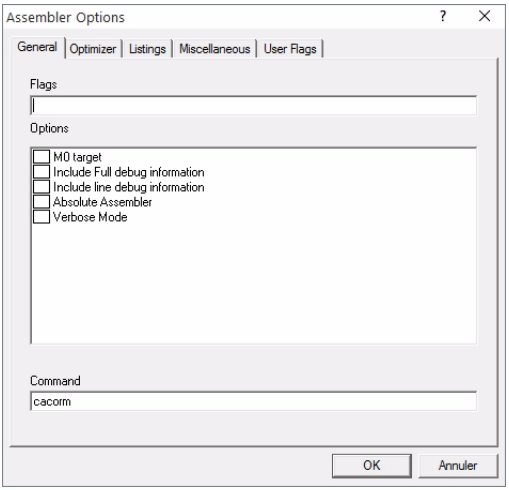


Figure 13: Assembler Options dialog box

The **Assembler Options** dialog box five tabs:

- **General options**
- **Optimizer options**
- **Listings options**
- **Miscellaneous options**
- **User Flags**

© 2016 COSMIC Software

Choose a tab and select the desired options. Selected options are displayed in bold and unselected options are greyed out. To select an option, simply click on the option description. To deselect an option, click on the option again.

The default assembler options can be overridden by setting assembler options for the individual source files. Refer to “Source File Options” on page 21 for details.

Setting default linker options

The **Linker** component lets you set the default ***clnk* utility** options that are used to link all files in a project. You can also specify a linker command file and edit the file.

Right click on the **Linker** icon to view a menu containing linker commands.

If you have set the automatic linker mode and vector files, the linker commands are described as following:

Options	Open the **Link Configuration** dialog box.
View Command File	Open the project link command file for viewing only.
Command File Automatic	Not available.
Command File NOT Automatic	Open the **Select Linker Command File** dialog box for editing.

By default or for manual linker mode, the linker commands are:

Options	Open the **Link Configuration** dialog box.
Edit Command File	Open the project link command file for editing.
Command File Automatic	Click to create an automatic link file. Then right clicking on the linker icon, you will get the linker commands described as above.
Command File NOT Automatic	Not available.

You can also click on the **Linker** ⊞ icon to display the **Linker Options** icon and the **Linker Command File** icon.

Setting the linker configuration

Select **Options** from the **Project Linker** menu (or right-click on the **Linker Options** icon) to open the **Link Configuration** dialog box.

The **Link Configuration** dialog box lets you specify:

- **Linker options**
- **Libraries path option**
- **Reporting mode option**
- **Memory banking option**

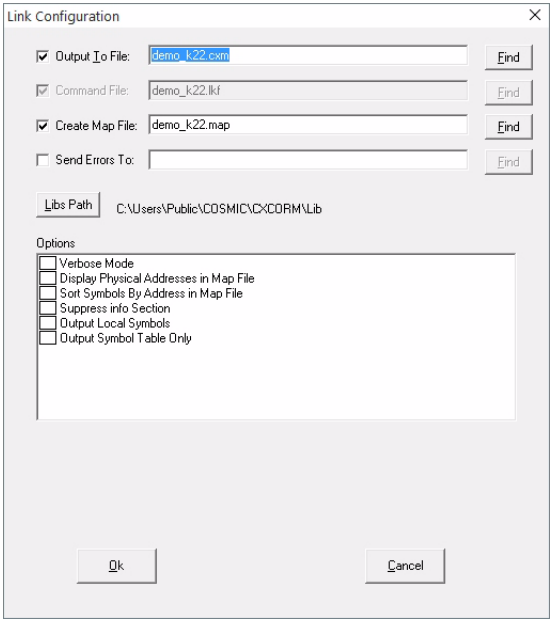


Figure 14: Link Configuration dialog box

© 2016 COSMIC Software

Specifying linker options

The Link Configuration dialog box lets you specify ***clnk* utility** options. These options are described in the following Table.

Output file	**(-o** option**)**: writes output to the specified file. This option is required and has no default value.
Command file (.lkf)	The linker command file. This option is required and has no default value.
Map file	**(-m** option**)**: produces map information for the program being built to the specified file.
Error file	**(-e** option**)**: logs errors in the text file specified instead of displaying the messages on the screen.

After you select any one of these files, you can click on the **Find** button to specify the file name and path.

Specifying the libraries path

Click on the **Libs Path** button to open the **Libraries Path Editor** then click on the **Select** button and set a path to the compiler library (**-l >** option).

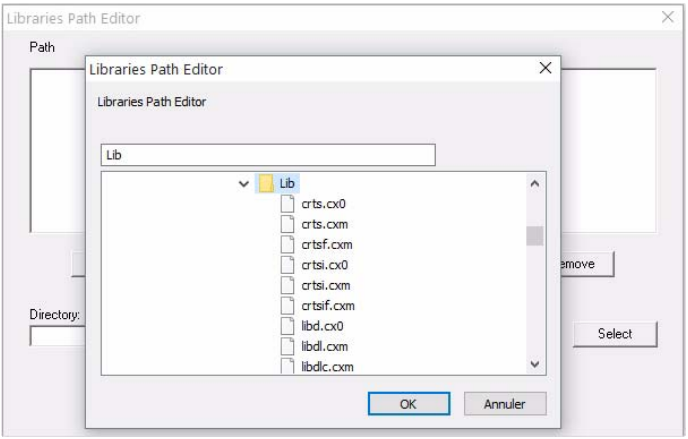


Figure 15: Libraries Path Editor

You can specify up to twenty library paths in any order. The paths are searched from top to bottom. After you add paths, they appear in order next to the **Libs Path** button.

Other linker options you can set are described in the following table.

Verbose mode	(**-v** option): be verbose.
Display Physical Addresses in Map file	(**-p** option): display symbols with physical address instead of logical address in the map file.
Sort Symbols per address in Map file	(**-sa** option): display symbols sort by address instead of alphabetic order in the map file.
Suppress info Section	(**-si** option): suppress the **.info.** segment content for compatibility with tools not supporting this segment yet.
Output Local Symbols	(**-sl** option): output local symbols in the executable file.
Output Symbol table only	(**-s** option): create an output file containing only an absolute symbol table, but still with an object file format.

Editing the linker command file

Before you can edit a linker command file, you must first check the **Command File** check box in the **Link Configuration** dialog box and then specify a linker command file name and path.

Select **Edit Command File** from the **Project Linker** menu (or right-click on the **Linker Command File** icon) to open the linker command file for editing.

© 2016 COSMIC Software

demo_k22.lkf (read only)

```
1   ##      link command file for K22FN512
2   ##      Copyright (c) 2015 by COSMIC Software
3   ##
4   +seg .vector -b 0x0 -m 0x400 -n .vector                  # vectors start address
5   +seg .sconst -b 0x410 -m 0xfbf0 -n .sconst -r2           # short constants follow vectors
6   +seg .const -a .sconst -m 0x7fbf0 -n .const              # large constants follow short constants
7   +seg .text -a .const -n .text                            # program follow constants
8   +seg .sdata -b 0x20000000 -o0 -m 0x1000 -n .sdata -r2 -id # short data start address
9   +seg .sbss -a .sdata -n .sbss                            # short bss follow short data
10  +seg .bss -a .sbss -o0 -m 0x10000 -n .bss                # large bss follow short bss
11  +seg .data -a .bss -n .data                              # large data follow large bss
12
13  ## interrupt vectors
14  "demo_k22_vector.o"
15
16  ## startup file
17  "crtsi.cxm"
18
19  ## application files
20  +inc "demo_k22.obl"
21
22  ## libraries
23  "libilc.cxm"                    # C library (if needed)
24  "libm.cxm"                      # machine library
25
26  ## symbols
27  +def __sram=pstart(.sbss)                        # bss start address
28  +def __eram=pend(.bss)                           # bss end address
29  +def __sdata=pstart(.sdata)                      # start of based data
30  +def __stack=0x20010000                          # stack pointer initial value
31  +def __memory=pend(.data)                        # symbol used by library
32
```

Figure 16: Linker command file for demok22.prjcxm

To edit the linker command file, you can make changes directly in the file using the options in the **Edit** menu. Type **Alt+E** to view the editing options. You can also right click to view a menu of editing options.

Changing the linker command file

Select **Change Command File** from the **Project Linker** menu to change the linker command file. The **Select Linker Command File** dialog box lets you specify a file name and path for the new linker command file.

After you select a new command file, the **Command File** check box is checked in the **Link Configuration** dialog box, and the linker command file name and path are displayed.

NOTE

In automatic mode, the linker command file and its content cannot be changed.

Specifying project builder utilities

The **Project Builder** component lets you specify utilities for building the project.

Right click on the **Project Builder** icon and select **Option** to open the **Builder Configuration** dialog box. You can also double-click on the **Project Builder** icon to display the **Builder Options** icon, and then right-click on the **Builder Options** icon to display the **Builder Configuration** dialog box.

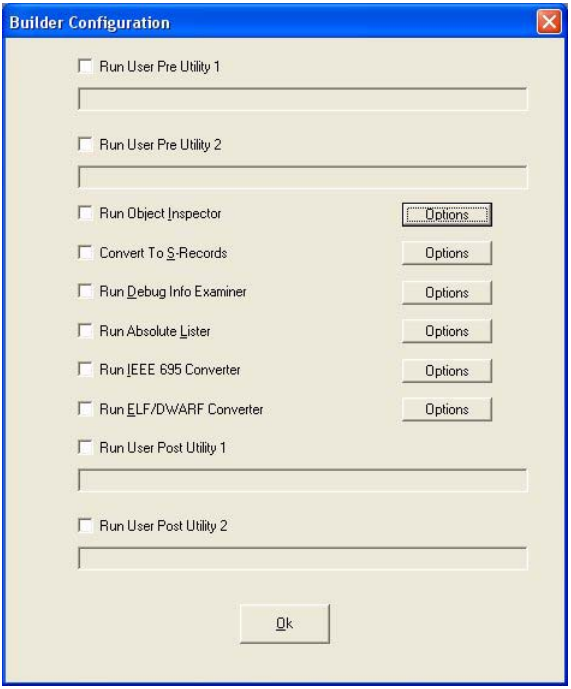


Figure 17: Builder Configuration dialog box

The **Builder Configuration** dialog box contains check boxes that let you specify which builder utilities to run. The builder utilities are described in the following table.

© 2016 COSMIC Software

Run Object Inspector	Runs the ***cobj* utility** to examine object modules. If you select **Run Object Inspector** and then click on the **Options** button, the **Options** dialog box appears. Refer to “Configuring the Object Inspector utility” on page 42 for details.
Convert to S-Records	Runs the ***chex* utility** to translate object module format to hexadecimal format. If you select **Convert to S-Records** and then click on the **Options** button, the **CHEX Configuration** dialog box appears. Refer to “Configuring the Hex Converter utility” on page 43 for details.
Run Debu Info Examiner	Runs the ***cprd* utility** to print debugging information about functions and data objects. If you select **Run Debug Info Examiner** and then click on the **Options** button, the **CPRD Configuration** dialog box appears. Refer to “Configuring the Debug Info Examiner utility” on page 45 for details.
Run Absolute Lister	Runs the ***clabs* utility** to generate absolute listings. If you select **Run Absolute Lister** and then click on the **Options** button, the **CLABS Configuration** dialog box appears. Refer to “Configuring the Absolute Lister utility” on page 47 for details
Run IEEE 695 Converter	Runs the ***cv695* utility** to generate IEEE695 format. If you select **Run IEEE 695 Converter** and then click on the **Options** button, the **cv695 Configuration** dialog box appears. Refer to “Configuring the IEEE695 Converter utility” on page 49 for details.
Run ELF/ Dwarf Converter	Runs the ***cvdwarf* utility** to generate ELF/Dwarf format. If you select **Run ELF/Dwarf Converter** and then click on the **Options** button, the **CVDWARF Configuration** dialog box appears. Refer to “The cv695 utility options are described in the following table.” on page 49 for details.
Run User Utility 1	Runs the specified user utility. You can specify a path and filename for the utility.
Run User Utility 2	Runs the specified user utility. You can specify a path and filename for the utility.

Configuring the Object Inspector utility

The ***cobj* utility** lets you inspect relocatable object files or executable output by the assembler or linker. The ***cobj* utility** can be used to check the size and configuration of relocatable object files or to output information from their symbol tables.

Right click on the **Object Inspector** icon and select **Options** to open the **Options** dialog box. You can also double-click on the **Object Inspector** icon to display the **Options** icon and then right-click on it to display the **Options** dialog box.

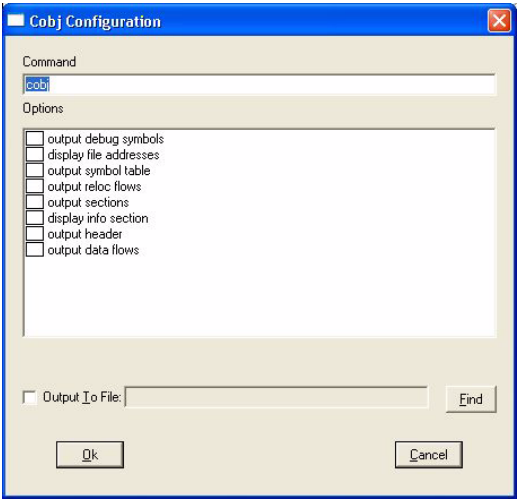


Figure 18: *cobj* utility Options dialog box

To select an option simply check the option checkbox (or wherever on the line) and it is added to the command line. To deselect an option uncheck it.

You can also specify a path and file name to receive the **Object Inspector** output. This file may be in relocatable format or executable format.

© 2016 COSMIC Software

Configuring the Hex Converter utility

The ***chex* utility** translates executable images produced by the ***clnk* utility** to one of several hexadecimal interchange formats.

Right click on the **Hex Converter** icon and select **Options** to open the **CHEX Configuration** dialog box. You can also double-click on the **Hex Converter** icon to display the **Options** icon and then right-click on it to display the **CHEX Configuration** dialog box.

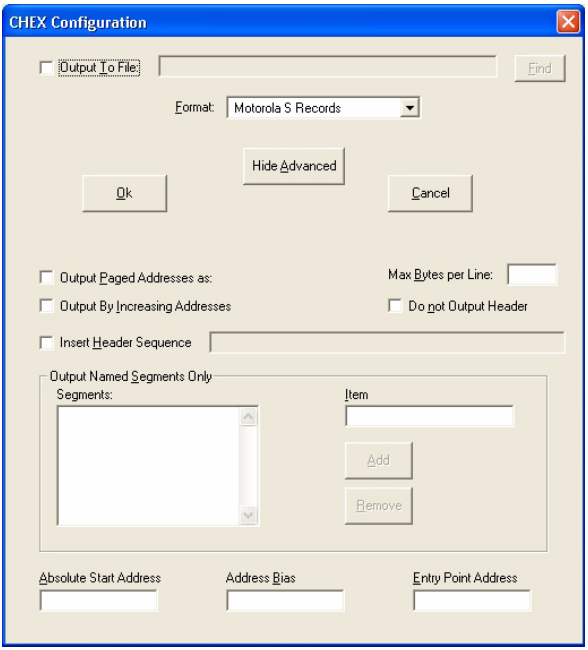


Figure 19: CHEX Configuration dialog box

The following Table describes the formats and options that are available.

Motorola S Records format	(**-fm** option) - produces S1 and S2 records as needed.
Motorola S2 Records format	(**-f2** option) - produces S2 records only. This is the default.
Motorola S3 Records format	(**-f3** option) - produces S3 records only. This is the default.
Intel Hex format	(**-fi** option) - produces Intel Hex only.
Absolute Start Address	(**-a** option) - the output address of the first byte.
Address Bias	(**-b** option) - subtract from any address before output.
Max Bytes per line	(**-m** option) - maximum data bytes per line. The default is 32 bytes per line.
Do not Output Header	(**-h** option) - do not output header sequence if such sequence exists for the selected format.
Output Paged Addresses	(**-p** option) - output addresses of banked segment.
Output by Increasing Addresses	(**-s** option) - sort the output addresses in increasing order.
Output to File	(**-o** option) - the default is STDOUT.
Insert Header Sequence	(**+h** option) - insert header sequence if such sequence exists for the selected format.
Output named segments only	(**-n** option). Up to twenty different named segments can be specified. To add a named segment to the **Segments** field, enter the named segment in the **Item** field and click on the **Add** button. To remove a named segment from the **Segments** field, select the segment and click on the **Remove** button.

© 2016 COSMIC Software

Configuring the Debug Info Examiner utility

The ***cprd* utility** extracts and prints information about functions and data objects from an object module or executable image that has been compiled with the **+debug** option.

Right click on the **Debug Info Examiner** icon and select **Options** to open the **CPRD Configuration** dialog box. You can also double-click on the **Debug Info Examiner** icon to display the **Options** icon and then right-click on it to display the **CPRD Configuration** dialog box.

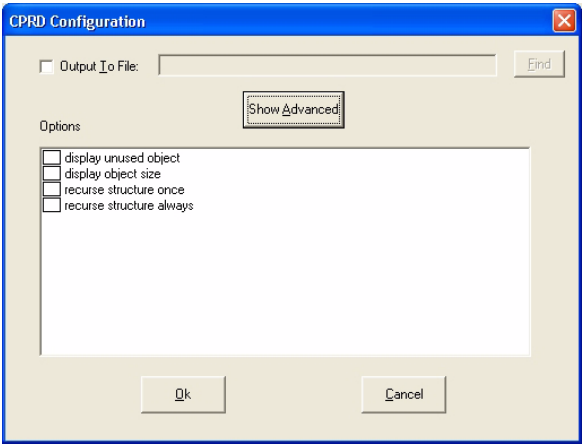
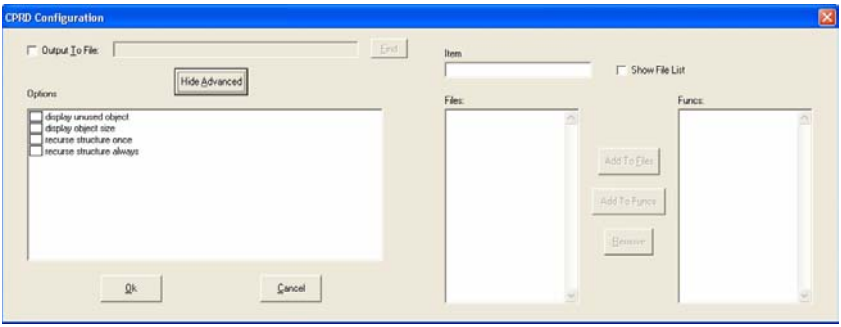


Figure 20: CPRD Configuration dialog box

display unused object	(**-u** option) - display only unused global variables.
display object size	(**-s** option) - display object size in bytes.
recurse structure once	(**-r** option) - display the first occurence of structure fields with their offset.
recurse structure always	(**-ra** option) - display any occurence of structure fields with their offset.

Click the **Show Advanced** button to get more options:



The **CPRD Configuration** dialog box lets you build a list of files and functions for debugging purposes. Enter a file or function name in the **Item** field, and then click on **Add to Files** to add the item to the **Files** list or **Add to Funcs** to add the item to the **Functions** list.

If you check the **Show File List** check box, the **Item** field changes to a **File List** field, with a drop-down list of the files in the project directory. Select a file from the list and then click on the **Add to Files** button to add it to the **Files** list.

To remove an item from either list, select the item and then click on the **Remove** button.

Each file in the **Files** list is processed with the **-fl** option, which prints debugging information about the file. By default, the ***cprd*** utility prints debugging information on all C source files.

Each function in the **Functions** list is processed with the **-fc** option, which prints information only about the function. By default, the ***cprd*** utility prints debugging information on all functions in a file.

You can also specify a path and file name to receive the debugger output. This is equivalent to the ***cprd*** utility **-o** option. By default, the ***cprd*** utility writes debugging information to the terminal screen.

© 2016 COSMIC Software

Configuring the Absolute Lister utility

The ***clabs* utility** processes relocatable C and Assembly listing files with the associated executable file to produce absolute listings with updated code and address values.

Right click on the **Absolute Lister** icon and select **Options** to open the **CLABS Configuration** dialog box. You can also double-click on the **Absolute Lister** icon to display the **Options** icon and then right-click on it to display the **CLABS Configuration** dialog box.

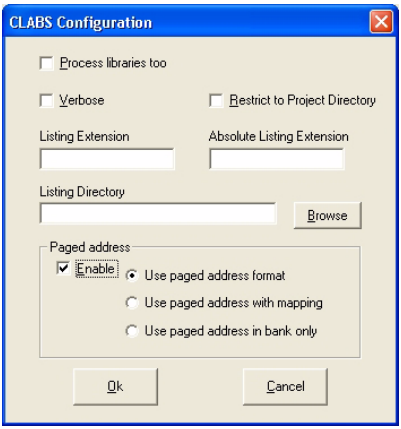


Figure 21: CLABS Configuration dialog box

The *clabs* utility options are described in the following table.

Process Libraries too	**(-a** option**) -** process also files located in libraries.
Verbose	**(-v** option**) -** the name of each module of the application is output to STDOUT.
Restrict to Project Directory	**(-l** option**) -** process files in the project directory only. The default is to process all files of the application.
Listing Extension	**(-r** option**) -** specify the input file extension. The default is "**.ls**".

Absolute Listing Extension	**(-s** option**) -** specify the output file extension. The default id "**.la**"
Paged Address	(**-p** option) - output addresses of banked segment.

© 2016 COSMIC Software

Configuring the IEEE695 Converter utility

The ***cv695* utility** converts a file produced by the linker into IEEE695 format.

Right click on the **IEEE695 Converter** icon and select **Options** to open the **CV695 Configuration** dialog box. You can also double-click on the **IEEE695 Converter** icon to display the **Options** icon and then right-click on it to display the **CV695 Configuration** dialog box.

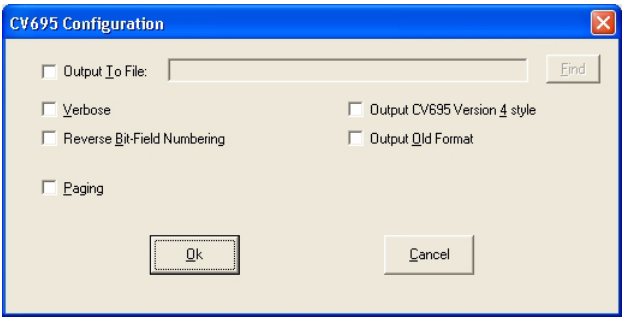


Figure 22: CV695 Standard Configuration dialog box

For some paged processor architecture, check the **Paging** option. The dialog box will be:

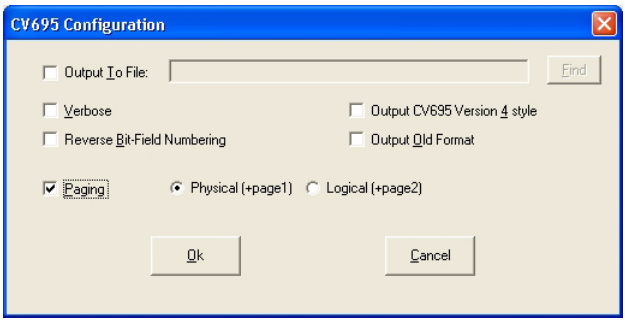


Figure 23: CV695 Paged Configuration dialog box

The *cv695* utility options are described in the following table.

Verbose	**(-v** option**) -** the ***cv695*** utility displays information about its activity.
Reverse Bit-Field Numbering	**(-rb** option**) -** reverses bitfield from left to right.
Paging	(**+page#** option) - this option is currently meaningful for the **HC12/HCS12** only. This option specifies the address format for bank-switched code. If you check the **Paging** check box, three options appear to the right: **Physical (+page1)** - the application is banked and the *cv695* utility outputs physical addresses. This is the default if **Paging** is checked. **Logical (+page2)** - the application is banked and the *cv695* utility outputs addresses in paged mode: `<page><offset_in_page>`. This is equivalent to the old **+paged** flag. **data paging (+dpage)** - the application uses data paging.
Output to File	(**-o** option) - you can specify a path and file name to receive the *cv695* utility output. By default, the *cv695* utility outputs to the file whose name is obtained from the input file by replacing the filename extension with "**.695**".

© 2016 COSMIC Software

Configuring the CVDWARF Converter utility

The ***cvdwarf* utility** converts a file produced by the linker into ELF/Dwarf format.

Right click on the ELF/DWARF Converter icon and select **Options** to open the CVDWARF Configuration dialog box. You can also double-click on the ELF/DWARF Converter icon to display the Options icon and then right-click on it to display the CVDWARF Configuration dialog box.

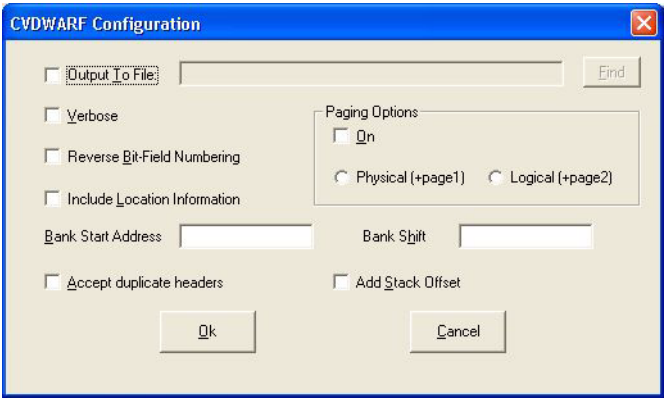


Figure 24: CVDWARF Configuration dialog box

The *cvdwarf* utility options are described in the following table:

Verbose	**(-v** option**) -** the ***cvdwarf*** utility displays information about its activity.
Paging	(**+page#** option) - this option is currently meaningful for the **HC12/HCS12** only. This option specifies the address format for bank-switched code. If you check the **Paging** check box, three options appear to the right: **Physical (+page1)** - the application is banked and the *cv695* utility outputs physical addresses. This is the default if **Paging** is checked. **Logical (+page2)** - the application is banked and the *cv695* utility outputs addresses in paged mode: `<page><offset_in_page>`. This is equivalent to the old **+paged** flag. **data paging (+dpage)** - the application uses data paging.
Reverse BitField Numbering	**(-rb** option**) -** reverses bitfield from left to right.
Include location information	(**-loc** option) location lists are used in place of location expressions whenever the object whose location is being described can change location during its lifetime. THIS POSSIBILITY IS NOT SUPPORTED BY ALL DEBUGGERS.
Output to File	(**-o** option) - you can specify a path and file name to receive the *cvdwarf* utility output. By default, the *cvdwarf* utility outputs to the file whose name is obtained from the input file by replacing the filename extension with "**.elf**".

© 2016 COSMIC Software

Specifying a project debugger

Right-click on the **Debugger** icon in the **Project>Tool** tab to open a dialog box that allows you to specify a debugger for the project.

After you select a debugger, the path and filename appears after the **Project Debugger** icon.

Once you have specified a debugger, you can double click on the **Debugger** icon to run the ZAP debugger with the project target file opened or select the debugger icon from the toolbar.

Building a Project

After a project is configured, you need to build the application. There are a three different ways to do this:

1) Right click on the **Project Name** icon in the Project window and select **Make** or **Build** from the pop-up menu.

2) Choose **Compile** (single, open file), **Make**, or **Build** from the **Project** pull-down menu.

3) Click on one of the following tools on the Tool bar:

	Compile tool - compiles (**.c** file) or assembles (**.s** file) an open project source file. Options are specified in the **Compiler** or **Assembler Options** dialog box.
	Build Project tool - checks source file up-to-date status and dependencies. Selectively compiles or assembles any out-of-date files and runs the Linker.
	Rebuild All tool - performs a Make by rebuilding all files regardless of their up-to-date status and then runs any utilities selected in the **Builder Configuration** dialog box.

For additional details on the project building tools, refer to Chapter 4, "*IDEA User interface*" in the IDEA User's Guide.

For additional details on building an IDEA project, refer to Chapter 7, "*Building an IDEA Project*" in the IDEA User's Guide.

© 2016 COSMIC Software

Debugging a project

IDEA lets you use a Cosmic ZAP debugger to debug your project. You can open the ZAP Debugger by clicking on the **Debugger** tool in the Tool bar.

NOTE

*Before you can use the ZAP debugger, you must first specify its location by right clicking on the **Debugger** tool in the **Tool Browser** (select **Tools** from the Main menu to open the **Tool Browser**). This opens a dialog box that allows you to specify the debugger for the project.*

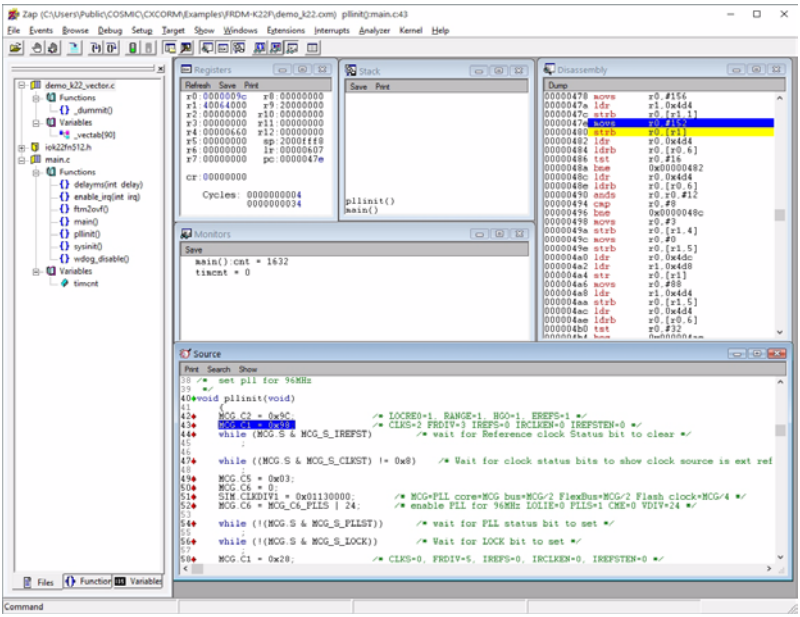


Figure 25: ZAP Debugger with project target file open

When you run the ZAP Debugger from within IDEA, the ZAP Debugger automatically opens the target file for the currently loaded project (for example, **demok22.cxm**).

For details on using the ZAP Debugger, refer to the ZAP User's Guide.